
EEM 561 Machine Vision

Lecture 6

Edge Detection

Spring 2019

Asst. Prof. Dr. Onur Kılınç

Eskisehir Tech. Uni., Dept. EEE

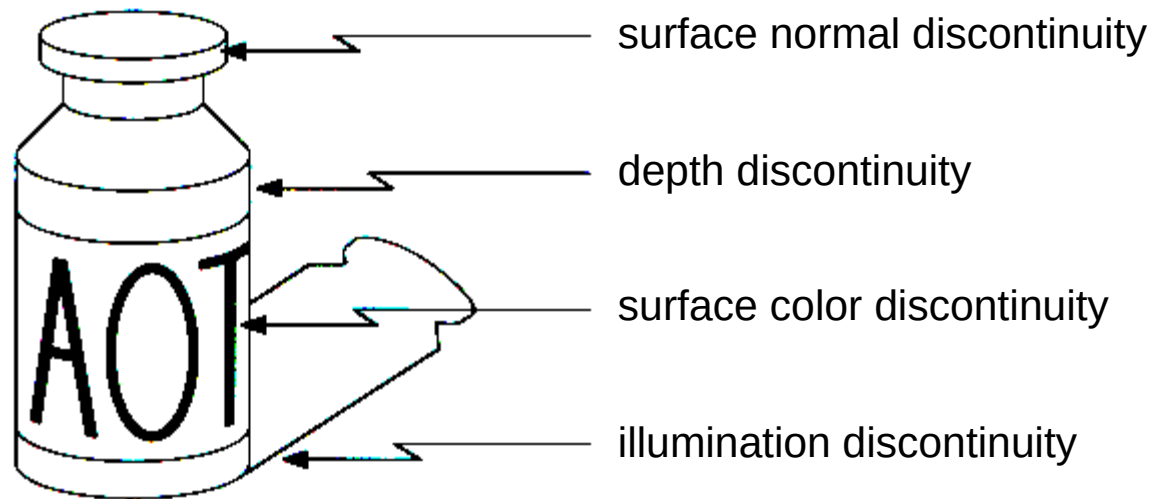
Edge detection



[Winter in Kraków photographed by Marcin Ryczek](#)

Edge detection

- **Goal:** Identify sudden changes (discontinuities) in an image
- Intuitively, edges carry most of the semantic and shape information from the image



Edge detection

- **Ideal:** artist's line drawing

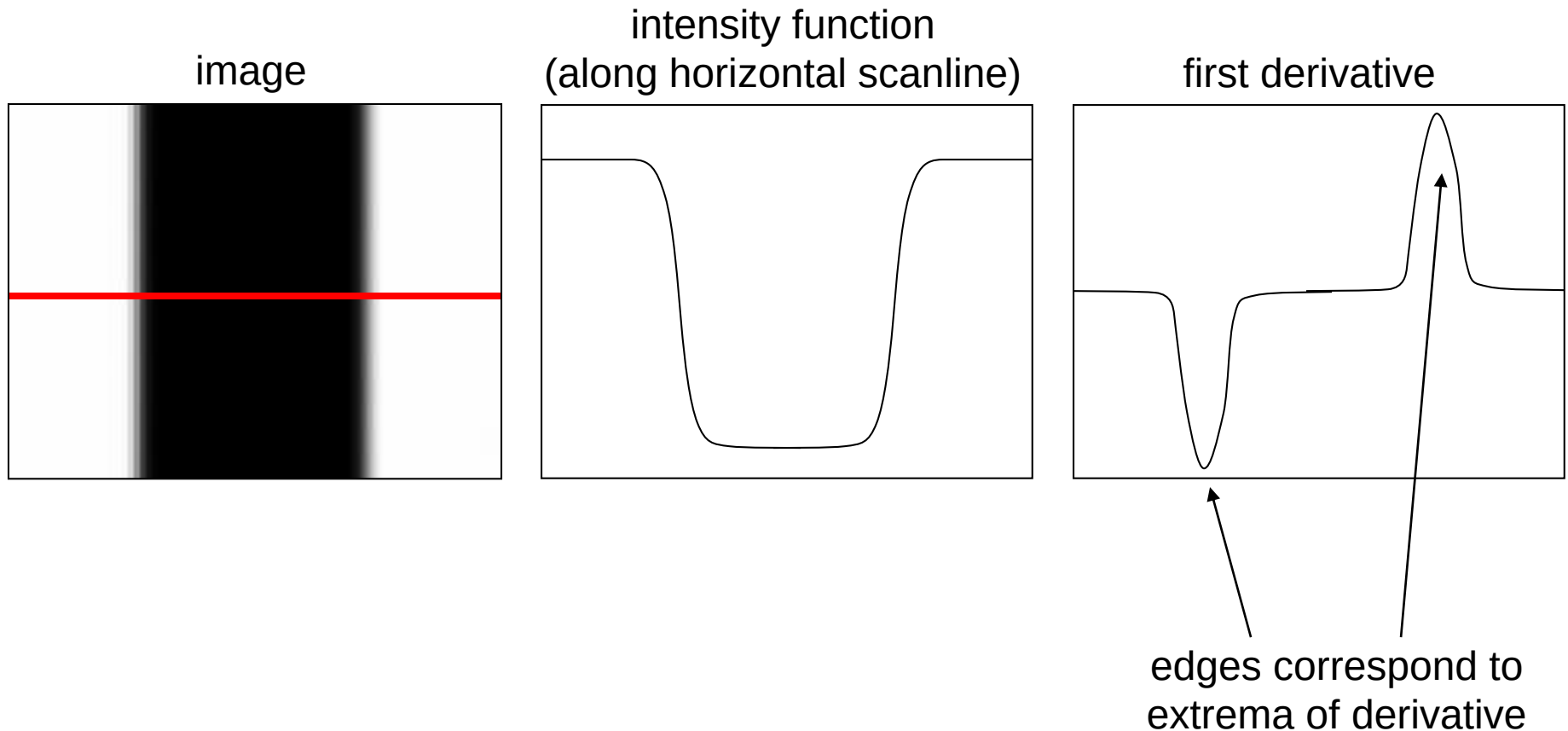


- **Reality:**



Edge detection

- An edge is a place of rapid change in the image intensity function



Derivatives with convolution

For 2D function $f(x,y)$, the partial derivative is:

$$\frac{\partial f(x, y)}{\partial x} = \lim_{\varepsilon \rightarrow 0} \frac{f(x + \varepsilon, y) - f(x, y)}{\varepsilon}$$

For discrete data, we can approximate using finite differences:

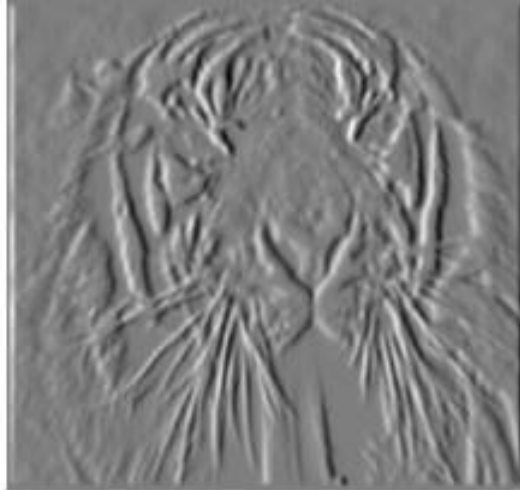
$$\frac{\partial f(x, y)}{\partial x} \approx \frac{f(x + 1, y) - f(x, y)}{1}$$

To implement the above as convolution, what would be the associated filter?

Partial derivatives of an image

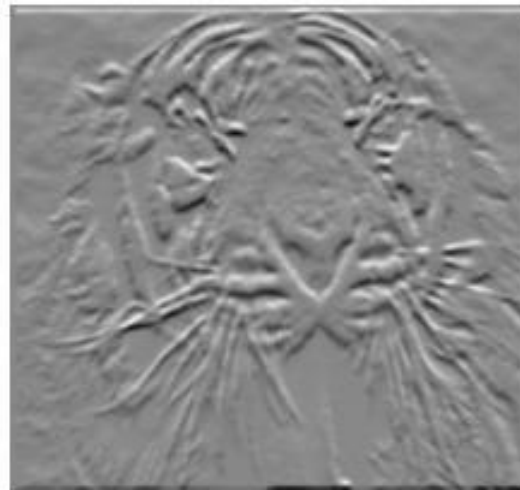


$$\frac{\partial f(x, y)}{\partial x}$$



-1	1
----	---

$$\frac{\partial f(x, y)}{\partial y}$$



-1	1
1	-1

 or

1	-1
-1	1

Which shows changes with respect to x?

Finite difference filters

Other approximations of derivative filters exist:

Prewitt: $M_x =$

-1	0	1
-1	0	1
-1	0	1

 ; $M_y =$

1	1	1
0	0	0
-1	-1	-1

Sobel: $M_x =$

-1	0	1
-2	0	2
-1	0	1

 ; $M_y =$

1	2	1
0	0	0
-1	-2	-1

Roberts: $M_x =$

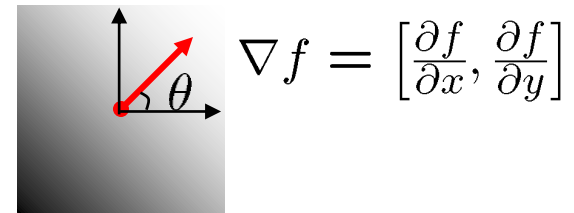
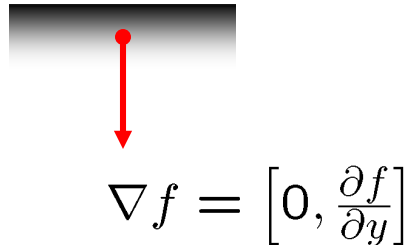
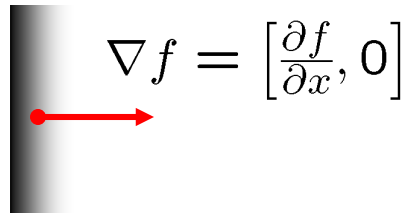
0	1
-1	0

 ; $M_y =$

1	0
0	-1

Image gradient

The gradient of an image: $\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$



The gradient points in the direction of most rapid increase in intensity

- How does this direction relate to the direction of the edge?

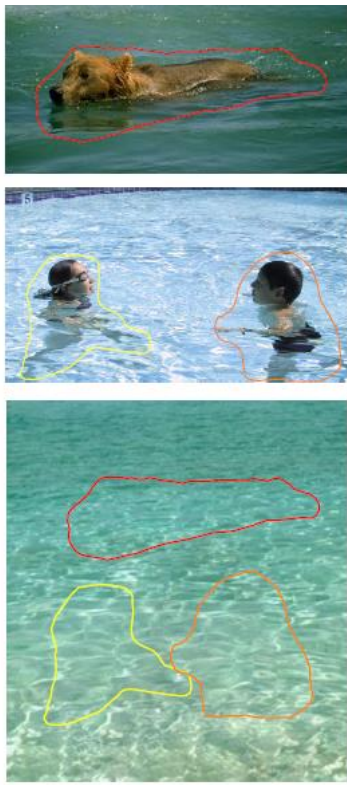
The gradient direction is given by $\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$

The edge strength is given by the gradient magnitude

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$

Application: Gradient-domain image editing

- Goal: solve for pixel values in the target region to match gradients of the source region while keeping background pixels the same



sources/destinations



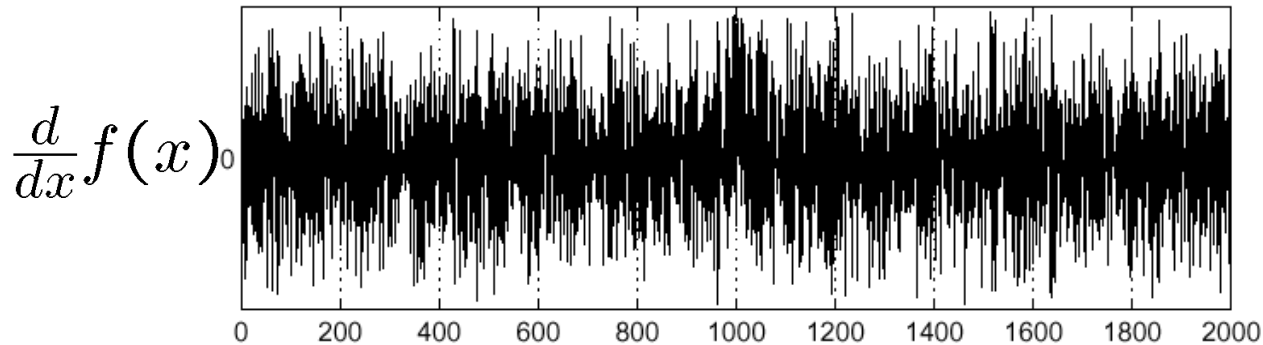
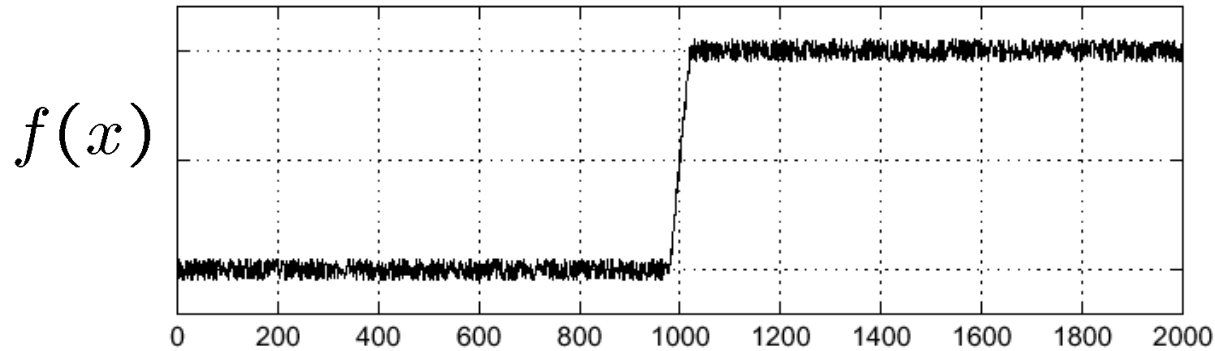
cloning



seamless cloning

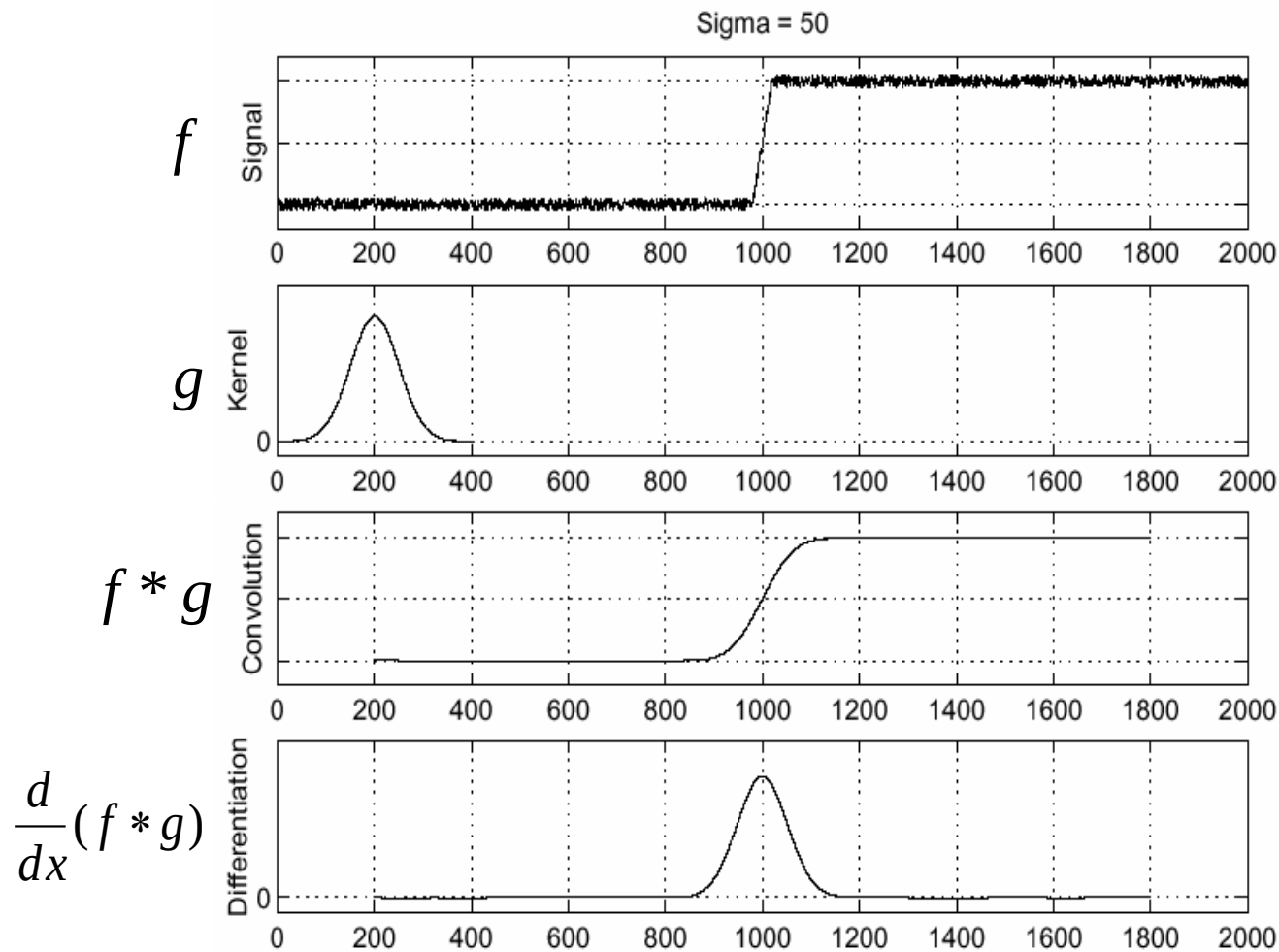
Effects of noise

Consider a single row or column of the image



Where is the edge?

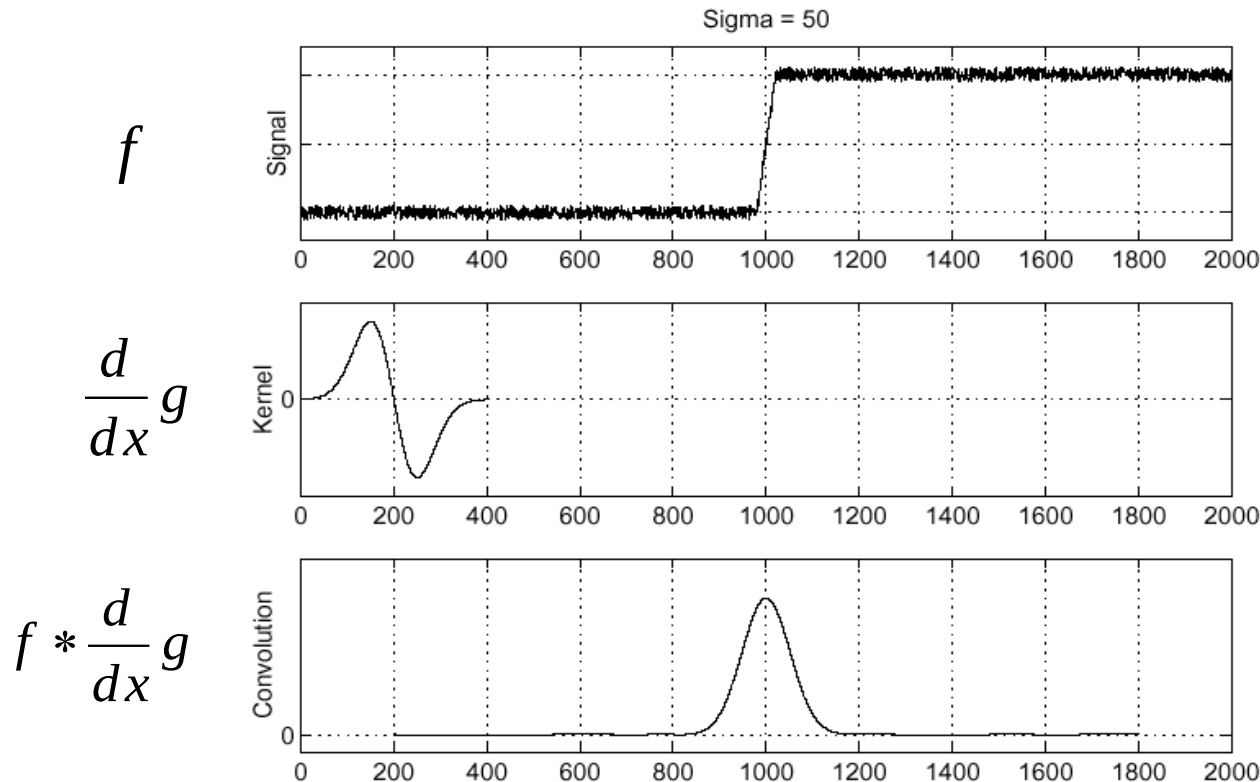
Solution: smooth first



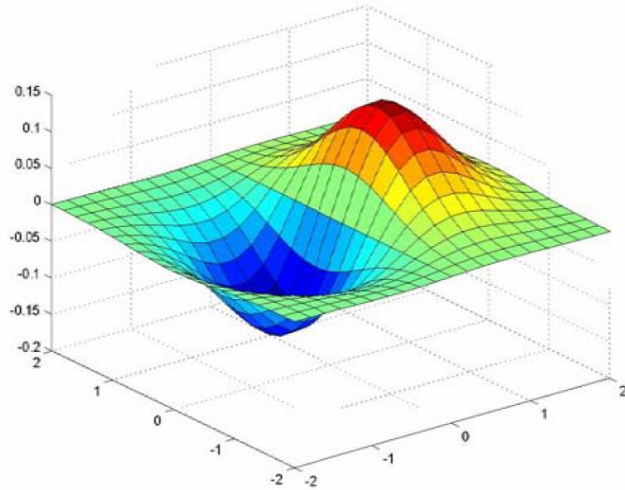
- To find edges, look for peaks in $\frac{d}{dx}(f * g)$

Derivative theorem of convolution

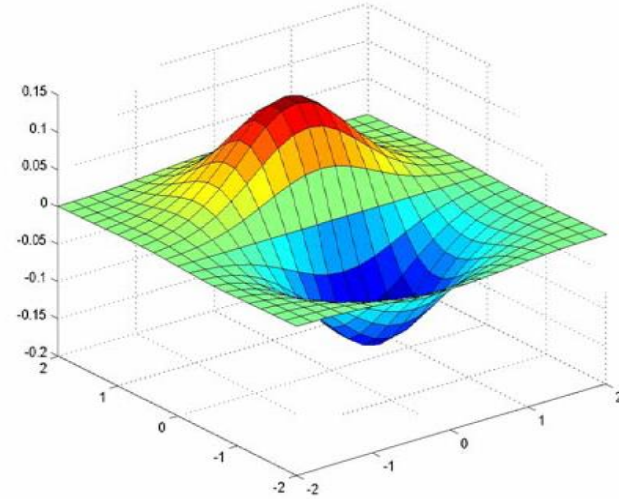
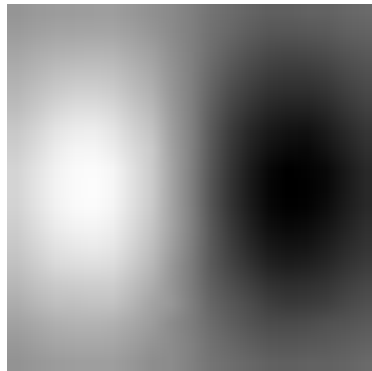
- Differentiation is convolution, and convolution is associative: $\frac{d}{dx}(f * g) = f * \frac{d}{dx}g$
- This saves us one operation:



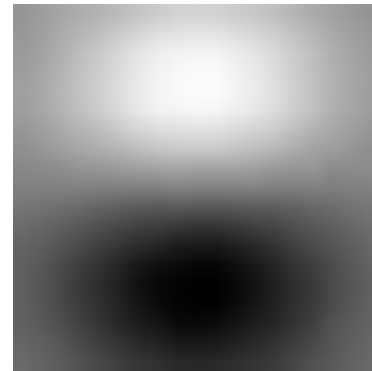
Derivative of Gaussian filters



x-direction

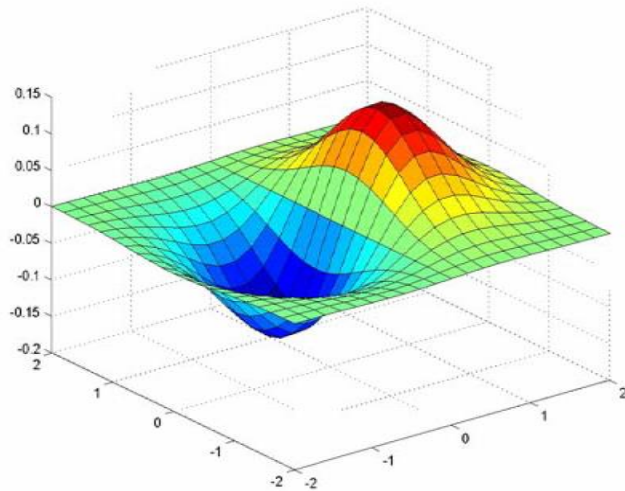


y-direction

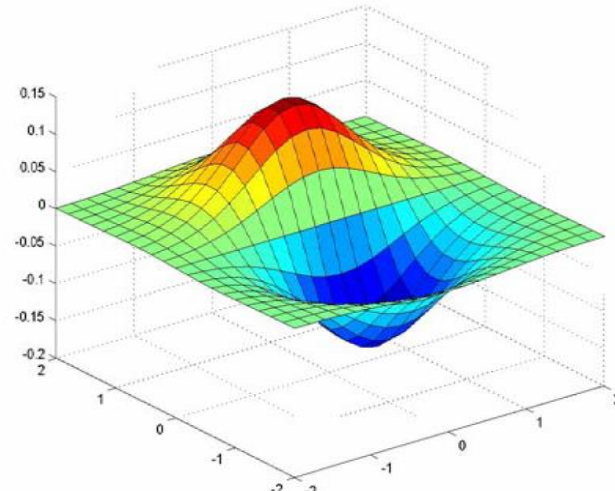
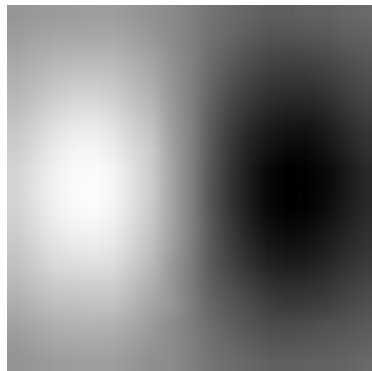


Which one finds horizontal/vertical edges?

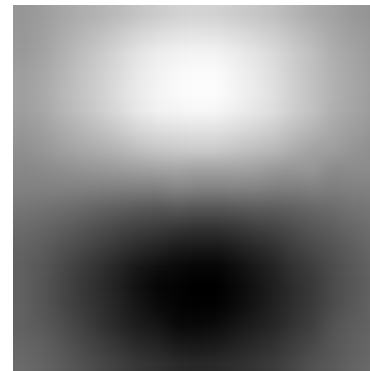
Derivative of Gaussian filters



x-direction



y-direction



Are these filters separable?

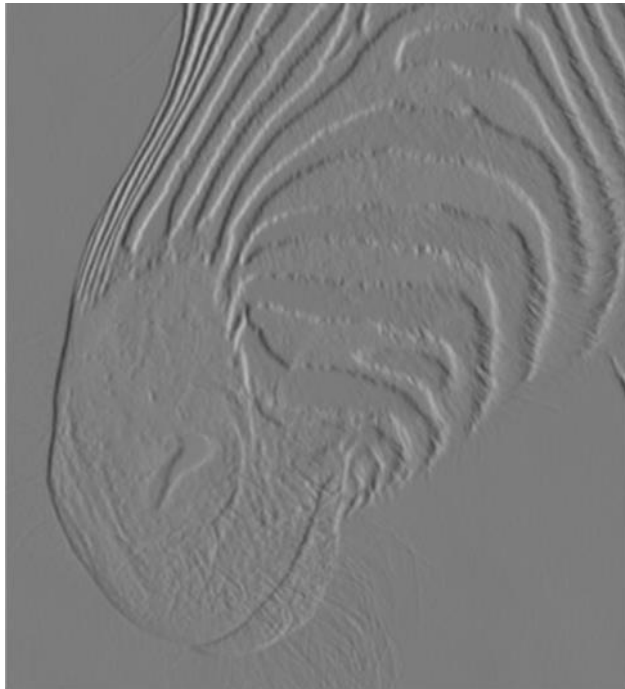
Recall: Separability of the Gaussian filter

$$\begin{aligned} G_{\sigma}(x, y) &= \frac{1}{2\pi\sigma^2} \exp^{-\frac{x^2 + y^2}{2\sigma^2}} \\ &= \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{x^2}{2\sigma^2}} \right) \left(\frac{1}{\sqrt{2\pi}\sigma} \exp^{-\frac{y^2}{2\sigma^2}} \right) \end{aligned}$$

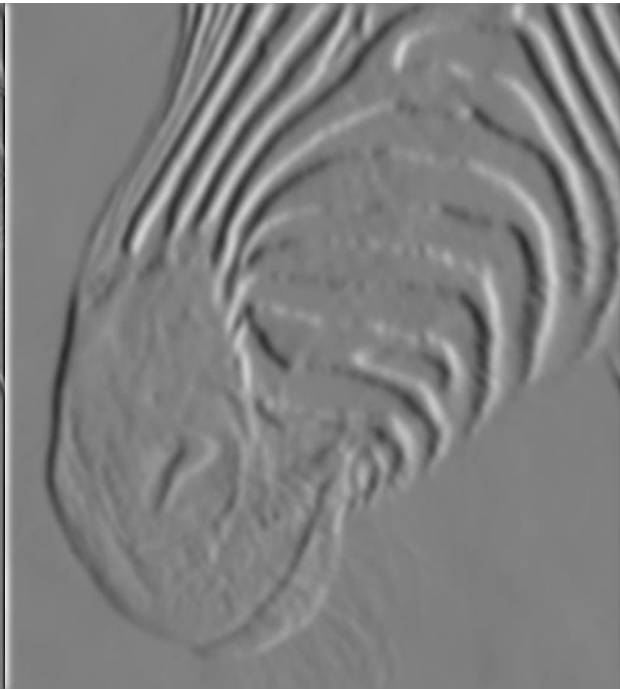
The 2D Gaussian can be expressed as the product of two functions, one a function of x and the other a function of y

In this case, the two functions are the (identical) 1D Gaussian

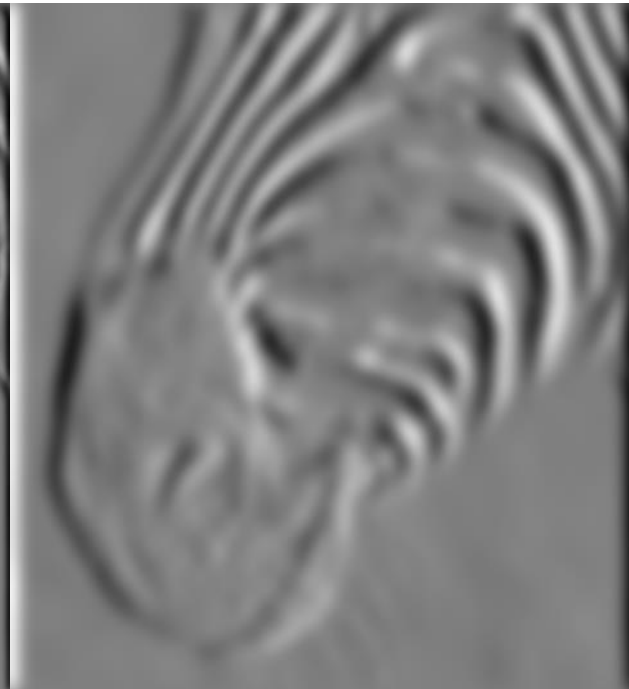
Scale of Gaussian derivative filter



1 pixel



3 pixels



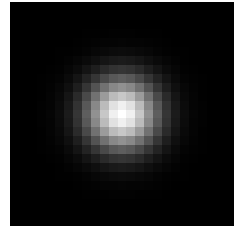
7 pixels

Smoothed derivative removes noise, but blurs edge. Also finds edges at different “scales”

Review: Smoothing vs. derivative filters

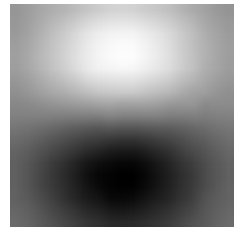
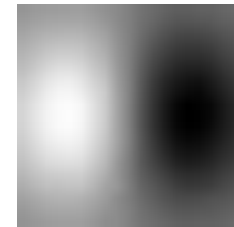
Smoothing filters

- Gaussian: remove “high-frequency” components; “low-pass” filter
- Can the values of a smoothing filter be negative?
- What should the values sum to?
 - **One:** constant regions are not affected by the filter



Derivative filters

- Derivatives of Gaussian
- Can the values of a derivative filter be negative?
- What should the values sum to?
 - **Zero:** no response in constant regions



Building an edge detector



original image



final output

Building an edge detector



norm of the gradient

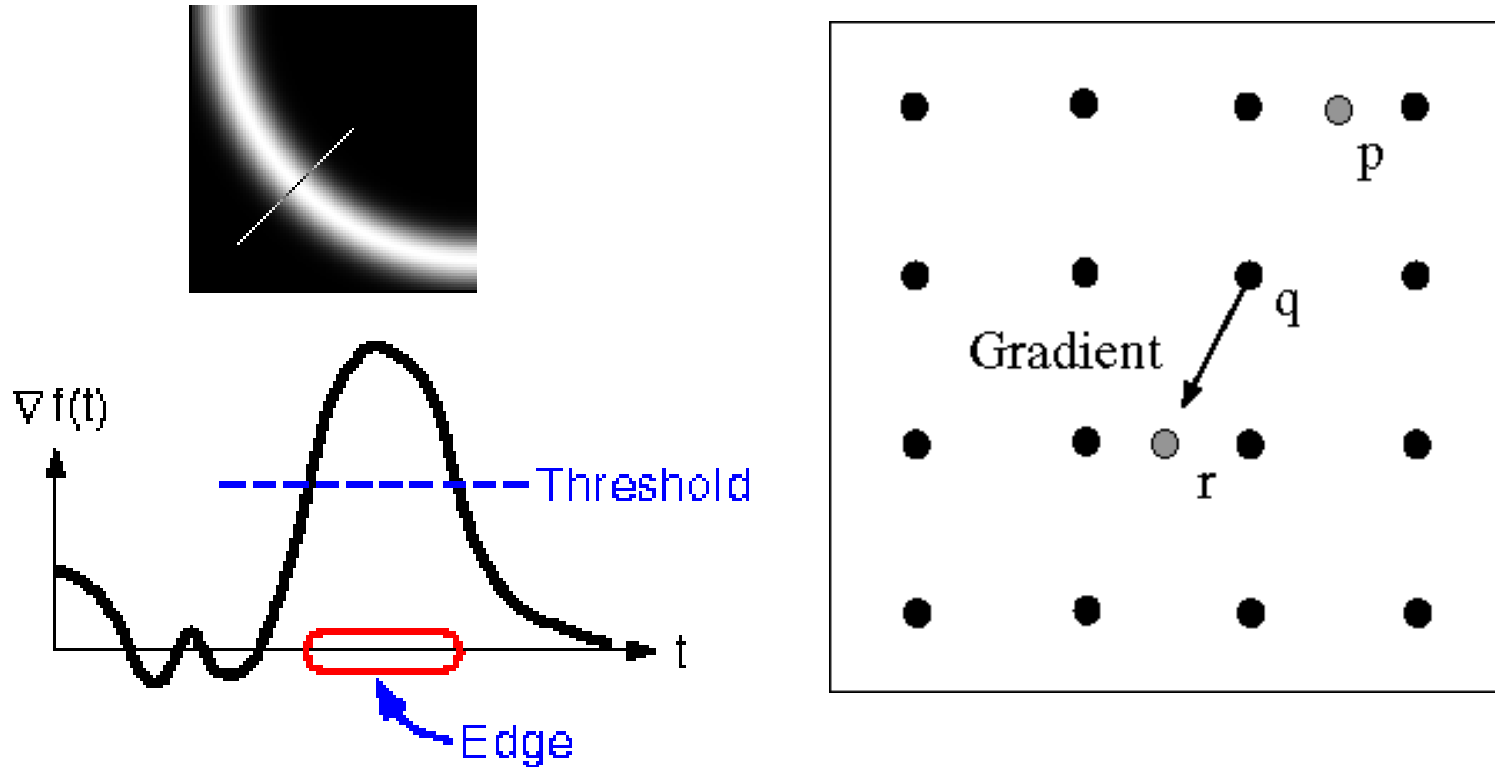
Building an edge detector



How to turn
these thick
regions of
the gradient
into
curves?

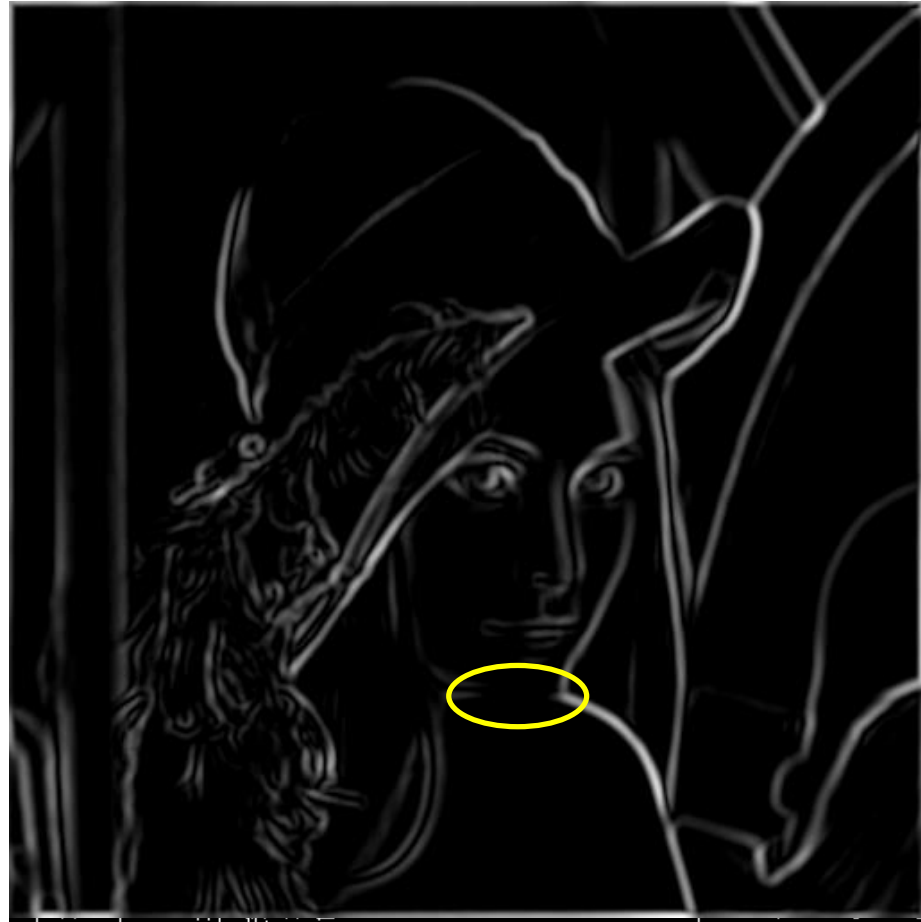
Thresholded norm of the gradient

Non-maximum suppression



- For each location q above threshold, check that the gradient magnitude is higher than at neighbors p and r along the direction of the gradient
 - May need to interpolate to get the magnitudes at p and r

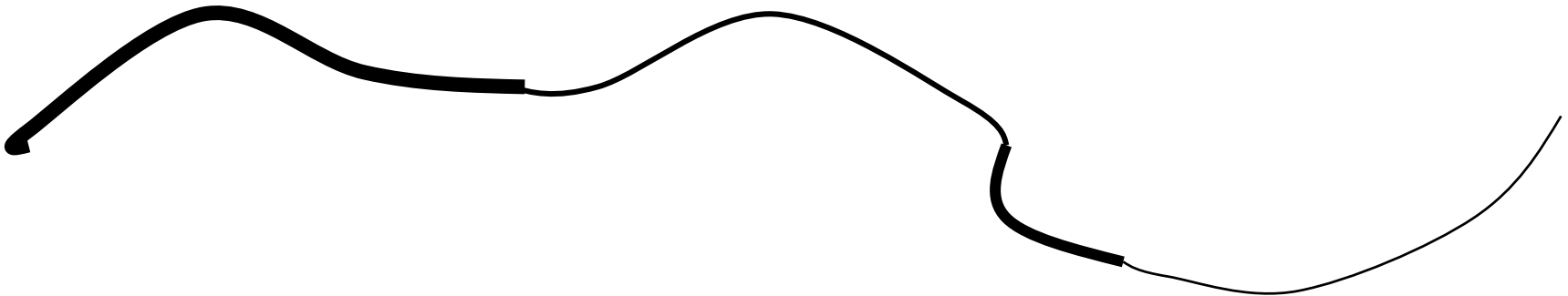
Non-maximum suppression



Another problem: pixels along this edge didn't survive the thresholding

Hysteresis thresholding

Use a high threshold to start edge curves, and a low threshold to continue them.



Hysteresis thresholding



original image



**high threshold
(strong edges)**



**low threshold
(weak edges)**



hysteresis threshold

Recap: Canny edge detector

1. Compute x and y gradient images
2. Find magnitude and orientation of gradient
3. **Non-maximum suppression:**
 - Thin wide “ridges” down to single pixel width
4. **Linking and thresholding (hysteresis):**
 - Define two thresholds: low and high
 - Use the high threshold to start edge curves and the low threshold to continue them

MATLAB: `edge(image, 'canny');`

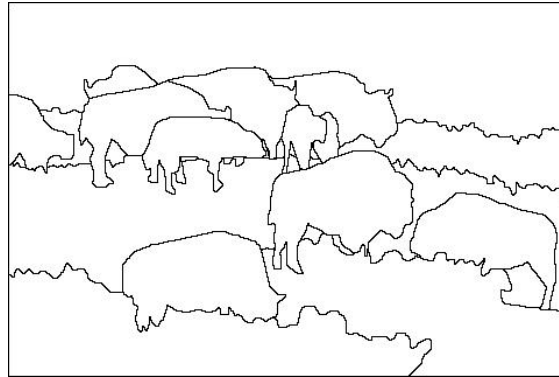
J. Canny, [A Computational Approach To Edge Detection](#), IEEE Trans. Pattern Analysis and Machine Intelligence, 8:679-714, 1986.

Image gradients vs. meaningful contours

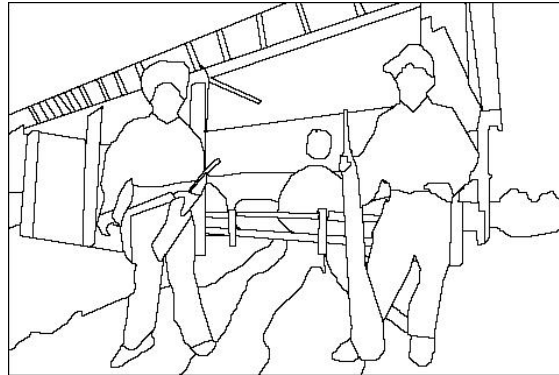
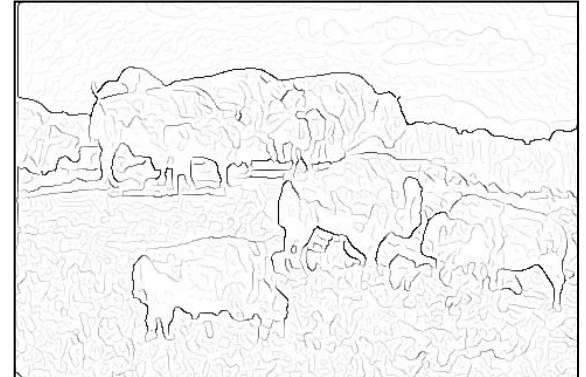
image



human segmentation



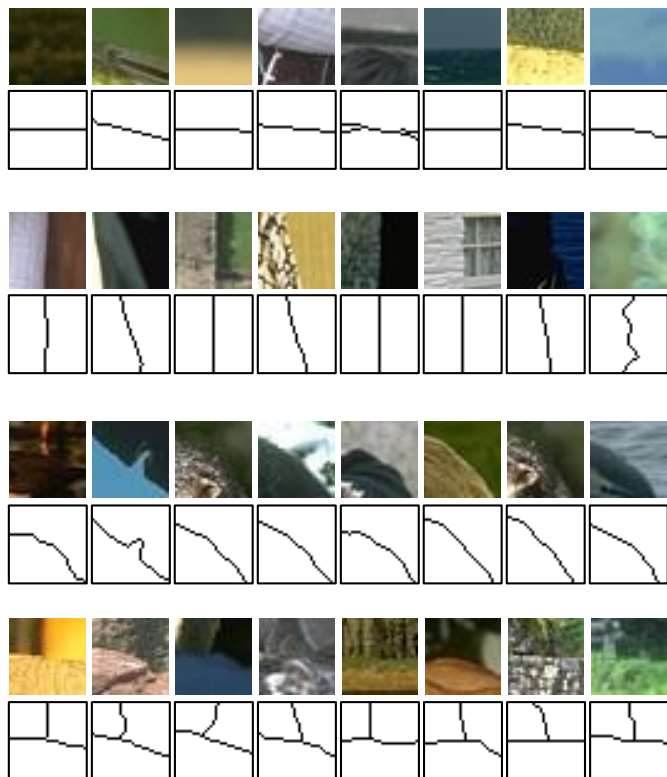
gradient magnitude



[Berkeley segmentation database](#)

Data-driven edge detection

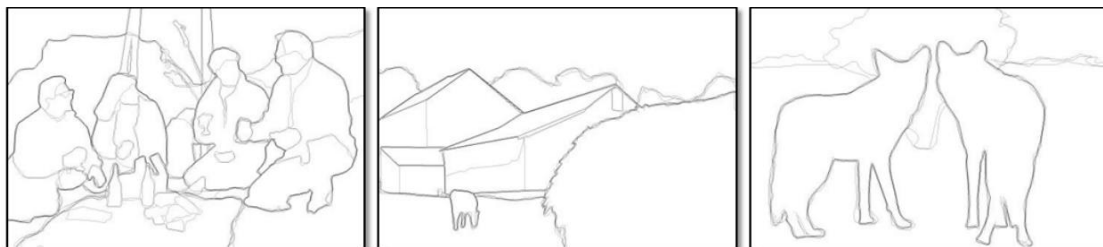
Training data



Input images



Ground truth



Output



Data-driven edge detection

